

数据结构三要素：
① 逻辑结构 $\begin{cases} \text{线性} \\ \text{非线性(树/图)} \end{cases}$
② 相关操作：增、删、改、查
③ 物理结构：(存储结构) $\begin{cases} \text{顺序(数组)} \\ \text{链式(链表, 带附加信息指针)} \\ \text{索引(大规模存储)} \\ \text{散列(哈希函数)} \end{cases}$

数据结构的抽象形式：

• 数据类型：一组性质相同的值的集合，以及定义于这个值集上的一组操作的总称。

$\begin{cases} \text{基本/内置数据类型: char, int, float, double, void} \dots\dots \\ \text{复合/派生数据类型: 数组、结构体、字符串、指针} \end{cases}$

• 抽象数据类型(ADT)：抽象数据组织及与之相应的操作(暂不考虑如何用物理结构实现)

eg: $\begin{cases} \text{线性: 栈, 队列, 列表} \\ \text{非线性: 字典、树、图} \end{cases}$

三要素: $\begin{cases} \text{数据对象} \\ \text{数据关系} \\ \text{操作} \end{cases}$

• C++面向对象：对象 + 类 + 继承 + 消息通信

* 算法：处理问题的策略 (算法 + 数据结构 = 程序)

。必须具备的特性：有输入(0个, 1个, ...)；有输出(1个, ...)；确定、有穷、能行。

* 算法的效率

1. 时间复杂度 (执行耗时)

① 事后估计 (引入 $\text{time}()$ 函数, 不推荐)

② 事前估计 (渐近分析, "抓大放小") eg: $T(n) = 2.7n^3 + 5.8n^2 + 5.3$, 则 $T(n) = O(n^3)$

* $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

嵌套程序段：从外向内计算, $T_z = T_1(n) \times T_2(m) = O(f(n) \times g(m))$

2. 空间复杂度 (占用内存大小) (同时间复杂度)